CHAPTER 6

Deep Q-Learning (DQN)

This chapter takes a deep dive into Q-learning combined with function approximation using neural networks. Q-learning in the context of deep learning using neural networks is also known as *Deep Q Networks* (DQN). The chapter first summarizes what I have talked about so far with respect to Q-learning. You will then look at code implementations of DQN on simple problems. Following this, you will take stock of Gymnasium and learn how it differs from OpenAI Gym. The next focus is on the Stable Baselines 3 (SB3) library and its associated ecosystem. You will also explore hyperparameter optimization using Optuna and plotting, both in the context of SB3. With the basic machinery in place, I will extend DQN to cover Atari game playing agent. The topics of DQN are followed by a quick exploration of various other RL environments and associated libraries covering the use of RL in financial market trading and in the field of robotics. However, be aware that these are still research environments useful to gain more insights. As some of these have multi-dimensional continuous value actions, there are better algorithms than DQN for training agents. This chapter focuses on learning about these environments and how to set them up. You will apply DQN where appropriate and detailed coverage of those algorithms appears in future chapters.

Finally, most of these algorithms will be covered using PyTorch, which in recent years has become the go-to deep learning framework displacing TensorFlow. TensorFlow has seen a steady decline over the last four to five years. The chapter includes an example in TensorFlow and, through that, you will realize that that big part of expertise in RL lies in problem framing and having a good understanding of the training algorithms. The role of deep learning framework is rather limited. Once you master the concepts using one framework, it is very easy to switch to another framework like TensorFlow or JAX.

© Nimish Sanghi 2024
N. Sanghi, *Deep Reinforcement Learning with Python*, https://doi.org/10.1007/979-8-8688-0273-7_6

Deep Q Networks

To introduce DQN, I first recap what you have learned so far. Chapter 4 talked about Q-learning as a model-free off-policy TD control method. You first looked at the online version, where you used an exploratory behavior policy (ϵ-greedy) to take a step (i.e., take action A) while in state S. The reward R and next state S' were then used to update the q-value $Q(S, A)$. Figure 4-16 explained the approach in pseudocode and Listing 4-6 showed the code implementing the same. The update rule, as shown in Equation 4-10, is reproduced here. You are encouraged to review the section "Q-Learning: An Off-Policy TD Control" in Chapter 4 before you proceed.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \cdot \left[R_{t+1} + \gamma \cdot \underset{A_{t+1}}{max} Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right] \tag{6-1}$$

I briefly talked about maximization bias and the approach of double Q-learning, wherein you use two tables of q-values. Following this, you looked at the approach of using a sample multiple times to convert online TD updates to batch TD updates, making it more sample efficient. This introduced the concept of a replay buffer. While it was only about sample efficiency in the context of discrete state and state-action spaces, with function approximation using neural networks, it becomes pretty much a must-have to make deep learning neural networks converge. You will revisit this again, and when I talk about prioritized replay, you will look at other options to sample from the replay buffer.

Moving along, in Chapter 5, you looked at various approaches of function approximation. You looked at tile encoding as a way to achieve linear function approximation. I then talked about DQN, that is, batch Q-learning with neural networks as function approximators. I went through a long derivation to arrive at a weight (i.e., neural network parameters) update equation, as given in Equation 5-25. That equation is reproduced here:

$$w_{t+1} = w_t + \alpha \,.\, \frac{1}{N} \sum_{i=1}^{N} \left[r_i + \gamma \max_{a_i'} \tilde{q}(s_i', a_i'; w_t^-) - \hat{q}(s_i, a_i; w_t) \right] \,.\, \nabla_{w_t} \hat{q}(s_i, a_i; w_t) \tag{6-2}$$

Note that I use the subscript i to denote the sample in the mini-batch and t to denote the time/loop index at which the weights are updated. Equation 6-2 is the one you will be using extensively in this chapter and the next. You will make various adjustments to this equation as you read about different modifications and study their impact.

I also talked about not having a theoretical guarantee of convergence under nonlinear function approximation with a gradient update. I have more to say about that in this chapter. The Q-learning approach was for discrete states and actions, where the update to the q-value was using Equation 6-1 as compared to adjusting the weight parameters for the deep learning–based approach in DQN. The Q-learning case had guarantees of convergence, while with DQN you had no such guarantee. DQN is computationally intensive as well. However, despite these shortcomings, DQN makes it possible to train agents using even raw images and not just specific observation vectors like position, velocity, angle, grid number, and so on. This is not at all conceivable in the case of plain Q-learning. Let's now put Equation 6-2 into practice to train the DQN agents in various environments.

Let's revisit the `CartPole` problem, which has a four-dimensional continuous state with values for the current cart position, velocity, angle of the pole, and angular velocity of the pole. The actions are of two types: push the cart to the left or push the cart to the right, with the aim of keeping the pole balanced as long as possible. This environment is part of OpenAI Gym library, which now has been forked to Gymnasium, as discussed earlier. The following are the details of the environment:

```
Observation:
    Type: Box(4)
    Num     Observation               Min             Max
    0       Cart Position             -4.8            4.8
    1       Cart Velocity             -Inf            Inf
    2       Pole Angle                -0.418 rad      0.418 rad
    3       Pole Angular Velocity     -Inf            Inf

Actions:
    Type: Discrete(2)
    Num         Action
    0           Push cart to the left
    1           Push cart to the right
```

You will build a small neural network with four as the input dimension since the state/observation vector is four dimensional, two hidden layers, followed by an output layer of dimension two, as the number of possible actions is two. There are two output q-values, that is $Q(S, A)$, one for each action. Figure 6-1 shows the network diagram.

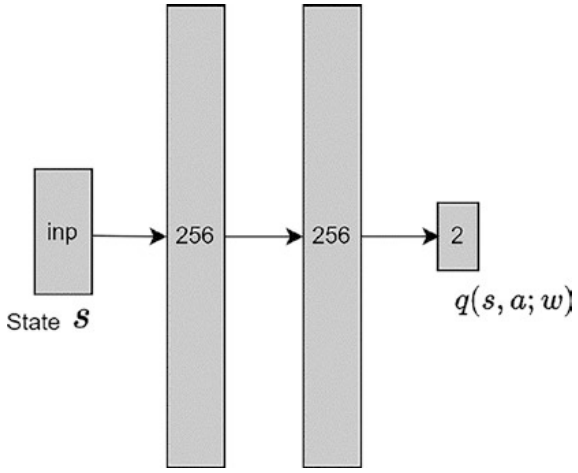


Figure 6-1. *The simple neural network*

There are many ways to build neural networks in PyTorch. The most flexible way is to sub-class the `torch.nn` module. In the `init` method of subclass, you define the building blocks like the linear layers, activation functions, and so on. Then you implement the `forward` method, the parent abstract method, which takes the input, passes it through the layers of the network defined in the `init` method, and returns the final output. The second method defines a list of all the layers in the sequence, all the way from input to output and passes that list to `nn.Sequential`. The second approach is a quicker one with minimal code, although it does not give you flexibility especially when it comes to multiple inputs, outputs, and skip connections.

This chapter uses the first approach, with the second approach embedded inside the `init` function, as shown in Listing 6-1. You also define a few additional convenience methods as part of the network. You will define three such methods. The first one is `get_qvalues` and it takes state/observation as a `numpy` array instead of a `torch` tensor that the `forward` expects. It then converts the `numpy` array to torch `tensor`, passes it through the `forward` method, and then again converts the returned PyTorch tensor from `forward` back to `numpy`. Such a method makes it easier to use the network inside the environment `step` function code, which otherwise would require you to convert the `tensor` to a `NumPy` or Python array while implementing the logic to step through the environment. Ideally these kinds of manipulations are best abstracted away and kept close to the network PyTorch code. If you use GPU, such a structure of the code makes it easier to handle the movement of batch of observations from CPU to GPU, process it through the network, and then shift it back to CPU making the back and forth move from CPU to GPU transparent to rest of the code. You can see this pattern in the first and third line of

code inside the `get_qvalues` function shown in Listing 6-1. While moving from `NumPy` to `tensor` you use the `device` argument to move the observations to the appropriate device that's hosting the neural network. In line three, you use the `Tensor.cpu()` chained call to move the q-values output by network back to CPU. The `get_qvalues` function takes a batch of states/observations as input, that is, a tensor of dimension $(N \times 4)$ where N is the number of samples. It passes the state values through the network to produce the q-values. The output vector has size $(N \times 2)$; one row for each input. Each row has two q-values, one for the left-push action and another for the right-push action.

The second function you'll implement is `sample_actions`. It implements the ϵ-greedy policy. It uses the `q-values` returned by the `get_qvalues` function as a `NumPy` array to choose the best action $(1 - \epsilon)$ fraction times and choose a random action ϵ fraction of times. It is what you use during learning to keep exploring the unvisited states. As discussed earlier you start with a high ϵ value like 0.6-0.8 and gradually reduce it down close to 0, with 0.05 being a popular choice. The function takes in a batch of q-values $(N \times 2)$. It uses the ε-greedy policy (equation 4-3) to choose an action. The output is the $(N \times 1)$ vector.

The third function, called `get_action`, is a truncated version of `sample_actions`. You use a greedy policy with no exploration. This is implemented in the `get_action` function. There is no random selection. In this function you deterministically and with zero exploration always return the best action with maximum q-value. As with `sample_actions`, the `get_action` function also takes a batch of q-values $(N \times 2)$ and produces $(N \times 1)$ action vector, one action per N observations in the batch. This function is used post-training to get the best action for a given state.

Listing 6-1 shows the code in PyTorch with the full code available in the `6.a-dqn-pytorch.ipynb` notebook.

Listing 6-1. DQN Network from `6.a-dqn-pytorch.ipynb`

```
class DQNAgent(nn.Module):
    def __init__(self, state_shape, n_actions, epsilon=0):

       #... boiler plate code omitted
       # ...
       # a simple NN
        self.network = nn.Sequential()
        self.network.add_module('layer1', nn.Linear(state_dim, 256))
```

```
        # code omitted ---- all other layers of network come here
        self.network.add_module('layer3', nn.Linear(256, n_actions))
        self.parameters = self.network.parameters

    def forward(self, state_t):
        # pass the state at time t through the network to get Q(s,a)
        qvalues = self.network(state_t)
        return qvalues

    def get_qvalues(self, states):
        # input is an array of states in numpy and output is Qvals as
        numpy array
        states = torch.tensor(np.array(states), device=device, dtype=torch.
        float32)
        qvalues = self.forward(states)
        return qvalues.data.cpu().numpy()

    def get_action(self, states):
        states = torch.tensor(np.array(states), device=device, dtype=torch.
        float32)
        qvalues = self.forward(states)
        best_actions = qvalues.argmax(axis=-1)
        return best_actions

    def sample_actions(self, qvalues):
        # sample actions from a batch of q_values using epsilon
        greedy policy
        epsilon = self.epsilon
        batch_size, n_actions = qvalues.shape
        random_actions = np.random.choice(n_actions, size=batch_size)
        best_actions = qvalues.argmax(axis=-1)
        should_explore = np.random.choice(
            [0, 1], batch_size, p=[1-epsilon, epsilon])
        return np.where(should_explore, random_actions, best_actions)
```

Note While not essential, you will gain more from the code discussions if you have some prior knowledge of PyTorch or TensorFlow. You should be able to create basic networks, define loss functions, and carry out basic training steps for optimization.

Next, you look at the way a similar structure is created in TensorFlow. TensorFlow's eager mode is very much like PyTorch. In TensorFlow, you have a similar sequential model that's available under `tf.keras.models.Sequential`. In TensorFlow, you have to implement a `__call__` method instead of the `forward` method that you used in PyTorch. The rest of the code looks very similar and therefore I do not list the code here. Interested readers can refer to the `6.b-dqn-tensorflow.ipynb` notebook for full details.

The code for the replay buffer is simple. You have an array called `self.buffer` to hold the previous examples. The `add` function takes in `state, action, reward, next_state, and done`, the values from a single step/transition by the agent, and adds it as a tuple to the buffer. If the buffer has reached its full length, it discards the oldest transition/tuple to make space for the new addition. The `sample` function takes an integer `batch_size` and returns `batch_size` samples/transitions from the buffer. In this vanilla implementation, each transition stored in the buffer has equal probability of getting sampled. Listing 6-2 shows the code for the replay buffer from the `6.a-dqn-pytorch.ipynb` notebook.

Listing 6-2. Replay Buffer (Same in PyTorch or TensorFlow) from `6.a-dqn-pytorch.ipynb`

```
class ReplayBuffer:
    def __init__(self, size):
        self.size = size #max number of items in buffer
        self.buffer =[] #array to hold buffer
        self.next_id = 0

   #... boiler plate code omitted

    def add(self, state, action, reward, next_state, done):
        item = (state, action, reward, next_state, done)
        if len(self.buffer) < self.size:
           self.buffer.append(item)
```

```
        else:
            self.buffer[self.next_id] = item
        self.next_id = (self.next_id + 1) % self.size

    def sample(self, batch_size):
        idxs = np.random.choice(len(self.buffer), batch_size)
        samples = [self.buffer[i] for i in idxs]
        states, actions, rewards, next_states, done_flags =
        list(zip(*samples))
        return np.array(states), np.array(actions),
        np.array(rewards), np.array(next_states), np.array(done_flags)
```

Moving ahead, you have a utility function called `play_and_record` that takes in an `env` (e.g., `CartPole`), an `agent` (e.g., `DQNAgent`), an `exp_replay` (`ReplayBuffer`), the agent's `start_state`, and `n_steps` (i.e., number of steps/actions to take in the environment). The function makes the `agent` take `n_steps` number of steps, starting from the initial state `start_state`. The steps are taken based on the current ε-greedy policy that the agent is following using `agent.sample_actions` and records these `n_steps` transitions in the buffer. Refer to the notebook from Listing 6-1 (`6.a-dqn-pytorch.ipynb`) for the detailed code showing how this is implemented.

Next, you look at the learning process. You first build the expression for loss, L, that you want to minimize. It is the averaged squared error between the target value of the current state action using a one-step TD value and the current state value. As discussed in Chapter 5, you use a copy of the original neural network that has weights w^- (w with superscript $-$). You use the loss to calculate the gradient of the agent (online/original) network's weight w and take a step in the negative direction of the gradient to reduce the loss. Note that as discussed in Chapter 5, you keep the target network with weights w^- frozen and update these weights on a less frequent basis. To quote from the section on batch methods of DQN in Chapter 5:

Here you have used a different weight vector w_t^- to calculate the estimate of the target. Essentially, you have two networks, one called online with weights w, which is being updated as per Equation 5-24, and a second similar network called target network but with a copy of the weight called w^-. Weight vector w^- is updated less frequently, say after every 100 updates of online network weight w. This approach keeps the target network constant and allows you to use the machinery of supervised learning.

The loss function L is as follows:

$$L = \frac{1}{N}\sum_{i=1}^{N}\left[r_i + (1-\text{done}_i).\gamma.\max_{a'}\hat{q}\left(s_i', a_i'; w_t^-\right) - \hat{q}\left(s_i, a_i; w_t\right)\right]^2 \quad (6\text{-}3)$$

You take a gradient (derivative) of loss L with respect to w and then use this gradient to update the weights w of the online network. You update the weights in the negative direction of gradient, as you want to reduce the loss by adjusting the values of weight w. This concept is from calculus that the gradient (also known as the derivative or the slope) of a function with respect to its variable gives the direction of maximum increase and therefore by adjusting the w in the negative direction of the gradient, you walk toward a new w that offers the fastest decrease in the function value.

This concept translates to the following equations:

$$\nabla_{w_t} L = -\frac{1}{N}\sum_{i=1}^{N}\left[r_i + (1-\text{done}_i).\gamma.\max_{a'}\hat{q}(s_i', a_i'; w_t^-) - \hat{q}(s_i, a_i; w_t)\right]\cdot\nabla_{w_t}\hat{q}\left(s_i, a_i; w_t\right) \quad (6\text{-}4)$$

$\hat{q}\left(s', A; w^-\right)$ is calculated using target network whose weights are held constant and refreshed periodically from the agent learning network.

The target is given by following:

- Nonterminal state: $r_i + \gamma.\max_{a'}\hat{q}(s_i', a_i'; w_t^-)$
- Terminal state: r_i

The value of $done_i$ is 1 when the environment terminated as a result of the action a_i. In Equation 6-4, by multiplying with $(1 - done_i)$ you get the two values of the target depending on the state of $done_i$ flag.

The gradient $\nabla\hat{q}\left(s_i, a_i; w_t\right)$ is calculated by back-propagation of the loss outputted by neural network and this is one of the primary reasons for using deep learning frameworks like PyTorch and TensorFlow, which automate the calculation of these gradients and have made these frameworks so powerful.

The basic flow is to design the network to output $\hat{q}\left(s_i, a_i; w_t\right)$ and then write a loss function to calculate loss L as given in Equation 6-3, which uses the network output $\hat{q}\left(s_i, a_i; w_t\right)$ and the target values to calculate the average squared loss as per Equation 6-3. The implementation of Equation 6-4 is automatically done by deep learning frameworks with a simple call of something like `loss.backward()`. Once the gradient has been calculated you have the value as given in Equation 6-4. Next, you

adjust the current weights of the network with a scaled amount of this gradient. The scaling factor α is the learning rate and it controls the amount of adjustment you make in each update. Usually, you start with higher α early in the training phase and gradually reduce it down as training of network progresses.

$$w_{t+1} \leftarrow w_t - \alpha \nabla_w L \tag{6-5}$$

Listing 6-3 shows the code to calculate L, as discussed previously. This is done by the `compute_td_loss` function. It takes in a batch of `states, actions, rewards, next_states`, and `done_flags`. It also takes in the discount parameter γ as well as the agent/online and target networks. The function first converts the `NumPy` arrays to `PyTorch` tensors and moves them to a CPU or GPU device. Next the states tensor batch is passed through the `agent` network to output the `predicted_qvalues` for all actions. You choose the specific q-value for the action tensor batch a_i to produce $\hat{q}(s_i, a_i; w_t)$, shown in the code as tensor `predicted_qvalues_for_actions`. Next, you compute the `predicted_next_qvalues` for all actions by passing the batch of `next_states` through the target network. You then take a `max` over the actions to produce $\max_{a_i'} \hat{q}(s_i', a_i'; w_t^-)$, shown in the code as `next_state_values`. Next, you calculate the target values $r_i + (1 - \text{done}_i) . \gamma . \max_{a'} \hat{q}(s_i', a_i'; w_t^-)$ shown in the code as `target_qvalues_for_actions`. The last line of the code calculates the loss by taking a difference of `target_qvalues` and `predicted q_values`, squaring, adding, and then averaging over the batch to return the computed loss L.

Listing 6-3. Compute TD Loss in PyTorch from `6.a-dqn-pytorch.ipynb`

```
def compute_td_loss(agent, target_network, states, actions, rewards,
next_states, done_flags, gamma=0.99, device=device):

    # convert numpy array to torch tensors
    states = torch.tensor(states, device=device, dtype=torch.float)
    actions = torch.tensor(actions, device=device, dtype=torch.long)
    rewards = torch.tensor(rewards, device=device, dtype=torch.float)
    next_states = torch.tensor(next_states, device=device,
    dtype=torch.float)
    done_flags = torch.tensor(done_flags.astype('float32'),device=device,
    dtype=torch.float)
```

```
    # get q-values for all actions in current states
    # use agent network
    predicted_qvalues = agent(states)

    # compute q-values for all actions in next states
    # use target network
    predicted_next_qvalues = target_network(next_states)

    # select q-values for chosen actions
    predicted_qvalues_for_actions = predicted_qvalues[range(
        len(actions)), actions]

    # compute Qmax(next_states, actions) using predicted next q-values
    next_state_values,_ = torch.max(predicted_next_qvalues, dim=1)

    # compute "target q-values"
    target_qvalues_for_actions = rewards + gamma * next_state_values *
    (1-done_flags)

    # mean squared error loss to minimize
    loss = torch.mean((predicted_qvalues_for_actions -
                       target_qvalues_for_actions.detach()) ** 2)

    return loss
```

At this point, you have all the machinery needed to train the agent to balance the pole. But first you need to define some hyperparameters like `batch_size`, total training steps `total_steps`, and the rate at which exploration ϵ will decay. It starts at 1.0 and slowly reduces to 0.05 as the agent learns the optimal policy. You also define an `optimizer`, which takes in the parameters of the agent network, that is, weight vector w. This is provided by passing `agent.parameters()` to the constructor function of the optimizer. There are many optimizers and Adam is a popular choice. The name is derived from adaptive moment estimation. The optimizer is called Adam because it uses estimations of the first and second moments of the gradient to adapt the learning rate for each weight of the neural network. You can read more about it in the PyTorch and TensorFlow documentation. The other important training parameter that you set is the frequency the target network is updated-that is, network with weights w^{-}. In this case you set it to 100 using `refresh_target_network_freq = 100`, which means that for every 100 updates of weight w of agent network, you will update the target

network. This way the target network provides a stable target for weight w to converge as well as it is not too infrequent leading to target network weights becoming stale. It is a hyperparameter and in real life you will probably be required to tune this. You also define the frequency of logging the training loss for plotting purposes and the frequency with which the trained agent will be evaluated to assess the quality of the policy agent is learning. Finally, you define `max_grad_norm` to clip the gradient's values. This is again a standard practice in deep learning, and it is a hyperparameter. All of these parameters are shown in Listing 6-4.

Listing 6-4. Training Parameters from `6.a-dqn-pytorch.ipynb`

```
#setup some parameters for training
timesteps_per_epoch = 1
batch_size = 32
total_steps = 50000

#init Optimizer
opt = torch.optim.Adam(agent.parameters(), lr=1e-4)

# set exploration epsilon
start_epsilon = 1
end_epsilon = 0.05
eps_decay_final_step = 2 * 10**4

# setup some frequency for logging and updating target network
loss_freq = 20
refresh_target_network_freq = 100
eval_freq = 1000

# to clip the gradients
max_grad_norm = 5000
```

Now you are all set to start training the agent. Listing 6-5 shows the code. You first reset the environment to obtain the initial state. After this point you will run a loop for a number of times equal to `total_steps`—that is, the number of training steps that the agent will be trained for. This is one of the parameters defined in Listing 6-4. In the loop you first set the exploration ϵ as per the settings defined in Listing 6-4. You use a one-line helper function called `epsilon_schedule` to do this. The implementation of this function is shown in Listing 6-5. You use the `play_and_record` function discussed earlier to fill the

Replay Buffer shown in Listing 6-1. You use a hyper-parameter called `timesteps_per_epoch` to control the number of steps you will take in each step of training. Usually this is set to 1 so that each time you run a training step you add one more tuple of training data to the buffer. This way, as the agent learns and explores new parts of the *state*, *action* space, the replay buffer is updated. Next, you sample a batch of stored transitions from the replay buffer with the size of buffer controlled by the `batch_size` parameter. The batch of training data so collected is passed to the `compute_td_loss` discussed in Listing 6-3. The return value stored in variable `loss` is explained in Equation 6-3. Next, you call `loss.backward()` which almost magically calculates the gradient $\nabla_w L$, as defined in Equation 6-4. This is the power of deep learning frameworks, which implement automatic differentiation. As discussed, you clip the gradients to ensure that each individual gradient value in the vector of gradients $\nabla_w L$ is within your defined bounds. Next, you call `opt.step()` to run the weight update as defined in Equation 6-5. This completes the core code in the training loop. Further in the training loop, you have some minor housekeeping code. You need to zero the vector of gradient $\nabla_w L$ after the update of the weight vector w. You also need to refresh the target network as per the `refresh_target_network_freq` parameter. Finally, you have to log the training loss, determine the mean return per episode as part of validation, and plot all these values to show the progress of training. Listing 6-5 shows all the code discussed. Some of the housekeeping code has been omitted from the listing to keep the focus sharp on what you need to understand. Readers interested in exploring the full code can refer to the `6.a-dqn-pytorch.ipynb` notebook.

Listing 6-5. Main Training Loop from `6.a-dqn-pytorch.ipynb`

```
def epsilon_schedule(start_eps, end_eps, step, final_step):
    return start_eps + (end_eps-start_eps)*min(step, final_step)/final_step

state,_ = env.reset()
for step in trange(total_steps + 1):

    # reduce exploration as we progress
    agent.epsilon = epsilon_schedule(start_epsilon, end_epsilon, step,
    eps_decay_final_step)

    # take timesteps_per_epoch and update experience replay buffer
    _, state = play_and_record(state, agent, env, exp_replay, timesteps_
    per_epoch)
```

```
    # train by sampling batch_size of data from experience replay
    states, actions, rewards, next_states, done_flags = exp_replay.
    sample(batch_size)

    # loss = <compute TD loss>
    loss = compute_td_loss(agent, target_network,
                           states, actions, rewards, next_states,
                           done_flags,
                           gamma=0.99,
                           device=device)

    loss.backward()
    grad_norm = nn.utils.clip_grad_norm_(agent.parameters(), max_grad_norm)
    opt.step()
    opt.zero_grad()

    if step % loss_freq == 0:
        td_loss_history.append(loss.data.cpu().item())

    if step % refresh_target_network_freq == 0:
        # Load agent weights into target_network
        target_network.load_state_dict(agent.state_dict())

    if step % eval_freq == 0:
        # eval the agent
        mean_rw_history.append(evaluate(
            make_env(env_name), agent, n_games=3, greedy=True, t_max=1000)
        )

    # plotting code omitted
```

You now have a fully trained agent. You train the agent and plot the mean reward per episode periodically as you train the agent for 50,000 steps. In the left plot of mean return per episode in Figure 6-2, the x-axis value of 10 corresponds to the 10,000th step. You also plot the TD loss every 20 steps, and that's why you have the x-axis on the right plot going from 0 to 2500, that is, 0 to 2500x20=50,000 steps. Unlike supervised learning, the targets are not fixed. Keep the target network fixed for a short duration and update it periodically by refreshing the target network weights with the online network. Also, as discussed, nonlinear function approximation (neural networks) with off-policy learning

(Q-learning) and bootstrapped targets (the target network being just an estimate of the actual value and the estimate formed using the current estimates of other q-values) has no convergence guarantee. The training may see losses going up and exploding or fluctuating. This loss graph is counterintuitive as compared to the loss graphs in the usual supervised learning. Figure 6-2 shows the graphs from the training DQN.

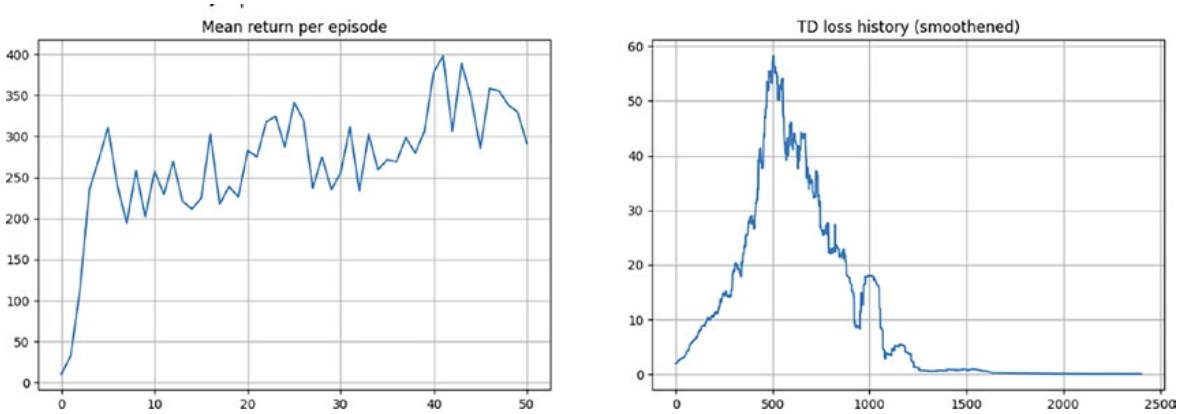


Figure 6-2. *Training curves for DQN*

I have more to say about other options for DQN training and logging using frameworks like Stable Baselines3 (SB3), RL Baselines3 Zoo (SB3 Zoo), weights and biases and others. However, before covering those issues, I take a quick detour to explain the origin of environment API from OpenAI and its forked version under Gymnasium. This understanding will be helpful, as some of the environment libraries and frameworks (like SB3, SB3 Zoo, and a few others) still follow an older version of API.

OpenAI Gym vs Farma Gymnasium

OpenAI made some breaking changes to the core API. It impacted workflows like how to create environments, what value is returned when you reset the environment, how to set the random seed, what values are returned by step function, and so on. These changes were introduced initially in OpenAI Gym v0.25.0 in July 2022 as opt-in and breaking change in v0.26.0 in Sep 2022.[1] The details are as follows:

[1] https://github.com/openai/gym/releases

- Step: The step function was modified to return five values: `obs, reward, termination, truncation, and info`. Original return values were `obs, reward, done, and info.` The reason for this change was to make a distinction between termination and truncation. `done=True` did not distinguish between the environment terminating and the episode truncating. In most cases where time limit is not a factor, you can set `done = truncation or termination.` You can read more about it here.[2]
- Render: The Render API was changed such that the mode had to be specified during `gym.make` with the `render_mode` keyword, after which the render mode is fixed. You can refer to release comments of Gym v0.25.0.
- Reset info: The `Env.reset` function was changed to return two values (`obs` and `info`) versus the older API returning only `obs`.
- No Seed function: While `Env.seed` was a helpful function, this was almost solely used for the beginning of the episode and was hence added to `gym.reset(seed=...)`.

In October 2022, the role of maintenance and updates to Gym was moved over to a new non-profit organization called the Farma Foundation. The code was moved over to a new Python package named `Gymnasium`, which carried over the new version of the APIs introduced in OpenAI Gym 0.26.0.

However, the story does not end here. Some of the third-party environments and libraries like SB3 and RL SB3 Zoo use parts or full older version of the API. SB3 and SB3 Zoo use the older API for `step` and `reset`. They retain the new `render` method, requiring `render_mode` to be specified right in the `gymnasium.make` call.

There are other environment libraries that are still using the pre 0.25.0—the popular `gym v0.21` version of API. Depending on the environment frameworks you plan to use, you should read the documentation and make sure you understand the version the library supports and if the library is compatible with OpenAI Gym or Frama Gymnasium.

Gymnasium also provides some environment wrappers to wrap environments following the old API to become compatible with newer post v0.26 API. Key wrappers that serve this compatibility need are as follows:

[2] https://github.com/Farama-Foundation/gym-docs/pull/115

- The wrapper `gymnasium.wrappers.EnvCompatibility` wraps an environment following the older version to become compatible with the newer API.
- The wrapper `gymnasium.wrappers.StepAPICompatibility` can transform an environment from new the step API to the old and vice versa.
- The wrapper `gymnasium.wrappers.PassiveEnvChecker` is a passive environment checker wrapper that surrounds the `step`, `reset`, and `render` functions to determine whether they follow the Gymnasium API.

Recording Videos of Trained Agents

Now that you understand the API changes between Gym and Gymnasium, let's continue with the topic of DQN training. Having trained the agent and reviewed the performance by way of plots, next you want to generate the video of the trained agent in action. I talked about it briefly in Chapter 2. However, this time I dive a bit deeper, which will help you gain a better understanding of the Stable-Baselines3 (SB3) library. SB3 gives you a unified structure for all the popular RL algorithms. It has optimized implementations of most of the popular RL algorithms used for training agents. It has integration with various experiment sharing, tracking, and logging services, such as weights and biases, HuggingFace, and MLFlow. SB3 also provides vectorized environments. Vectorized environments stack multiple independent environments into a single environment. Instead of training an RL agent on one environment per step, vectorized environments allow you to train it on n environments per step. You will use two such vector environment classes—`DummyVecEnv` and `VecVideoRecorder`—to help you record the video of trained agent.

`DummyVecEnv` creates a simple vectorized wrapper for multiple environments, calling each environment in sequence on the current Python process. This can also be used for RL methods that require a vectorized environment, but that you want a single environment to train with. You will use the latter purpose of wrapping a single environment with `DummyVecEnv`. The reason is that the recording API expects a vectorized environment. Listing 6-6 shows the code for the `record_video` function. The first line of code creates an array with a single `CartPole` environment. You'll use `Gymnasium` (imported as `gym`) to call its `make` function and pass the array to `DummyVecEnv`'s `init` function.

Next, you wrap the vectorized environment with `VecVideoRecorder`, providing arguments such as the vectorized environment for which you want to record the video, the folder path where the video will be recorded, the trigger callback which starts the recording, and the name prefix to specify the prefix of the video file.

Once `vec_env` has been wrapped in `VecVideoRecorder`, you run the standard rollout of episode, starting from the reset of the environment with `obs = vec_env.reset()`. Note that the SB3 vectorized environment's `reset` function returns a single value—the observation that conforms to the older version of the Gym API. Next, you run a loop to step through the environment with the policy being the trained agent.

You extract the generated video filename so that it can be returned by the `record_video` function and consumed by the `play_video` function. You can refer to the accompanying notebook for the implementation of the `play_video` function. Finally, you call the `vec_env.close()` function to close the environment, stop the video, and free up all the resources.

Listing 6-6. Video Recording Code from `6.a-dqn-pytorch.ipynb`

```
def record_video(env_id, video_folder, video_length, agent):

    vec_env = DummyVecEnv([lambda: gym.make(env_id, render_mode="rgb_
    array")])
    # Record the video starting at the first step
    vec_env = VecVideoRecorder(vec_env, video_folder,
                           record_video_trigger=lambda x: x == 0, video_
                           length=video_length,
                           name_prefix=f"{type(agent).__name__}-{env_id}")

    obs = vec_env.reset()
    for _ in range(video_length + 1):
        action = agent.get_action(obs).detach().cpu().numpy()
        obs, _, _, _ = vec_env.step(action)
    # video filename
    file_path = "./"+video_folder+vec_env.video_recorder.path.
    split("/")[-1]
    # Save the video
    vec_env.close()
    return file_path
```

Confusingly, Gymnasium also has a set of vectorized environments that follow the new API with a `step` function returning a batch of `observations, rewards, terminations, truncations,` and `infos`. Even the Gymnasium vectorized environment's `reset` function adheres to the newer version, returning a batch of observations and info from the vectorized environment. You could have implemented the same video recording code using the Gymnasium API. Interested readers can refer to the Gymnasium documentation for more details.

Next, you leverage the same code you saw in Chapter 2 with which you can push the trained agent and the network weights along with the video to HuggingFace. The code is similar to what you saw in Chapter 2. Inside the `6.a-dqn-pytorch.ipynb` notebook, you can look at the section named "Share the agent using HuggingFace" and explore the arguments passed to the `package_to_hub()` function call.

End-to-End Training with SB3

Up to this point, you handcrafted the agent network, the loss calculation, and the training code for the DQN algorithm. While this is good for learning, eventually in projects you want to leverage well implemented scalable DQN algorithms with all the bells and whistles. SB3 provides you with most of the popular algorithms with parameters to control the behavior. Let's look at the complete code to create and train the agent using SB3.

In terms of actual code, you first create the `CartPole` environment, and this code is similar to what you used earlier while handcrafting the DQN algorithm in earlier sections. Next, you create `policy_kwargs` to define the network layers and activation function to be used inside the neural network. You create a similar network like before with two hidden layers of 256 nodes each and ReLU as the activation function. There are multiple ways to define the network. The most basic is to go with the default and not create `policy_kwargs`, the policy configuration dictionary. This approach creates a default network of two hidden layers with 64 nodes each and ReLU activations. The most complex and flexible way is to create your own custom network similar to Listing 6-1. For details, refer to the documentation in SB3 under the "Custom Policy" topic.

Once `policy_kwargs` is defined, you instantiate the `model` with a call to the SB3 provided DQN class. It takes in `MlpPolicy` as the type of policy, `env` as the variable holding the environment, and `policy_kwargs` as the custom configuration of that policy. There are many other parameters you might need to tweak for your specific use case.

This pretty much completes the creation code. Next, you call `model.learn()` with parameters like training time steps, logging intervals, and flags to control the display of training progress. After the training is over, you save the model with a call to `model.save()`, providing the path where you want the model to be saved. The six-line code given in Listing 6-7 is sufficient to instantiate and train a DQN on the `CartPole` environment.

Listing 6-7. End-to-End DQN with SB3 from `6.a-dqn-pytorch.ipynb`

```
# create the DQN agent
from stable_baselines3 import DQN

# create the environment
env_name = env_name = 'CartPole-v1'
env = gym.make(env_name, render_mode="rgb_array")

#define a policy matching ours
#define the activation function and the network layers size
policy_kwargs = dict(activation_fn=torch.nn.ReLU,
                     net_arch=[256, 256])
model = DQN("MlpPolicy", env, policy_kwargs=policy_kwargs, verbose=1)

model.learn(total_timesteps=1e5, log_interval=500, progress_bar=True)
model.save("logs/6_a/sb3/dqn_cartpole")
```

End to End Training with SB3 Zoo

As discussed earlier, RL Baselines3 Zoo is a training framework for reinforcement learning (RL). It provides scripts for training, evaluating agents, tuning hyperparameters, plotting results, and recording videos. In addition, it includes a collection of tuned hyperparameters for common environments and RL algorithms, and agents trained with those settings.

To expose you to various options, this example uses weights and biases (`wandb`) integration to log these experiments. With `wandb`, you can capture the logging and share it with other team members. It also helps you track all your experiments, the specific parameters used in a given run, the training logs, and various other system level details. You first need to create a `wandb` account by going to `www.wandb.ai`. Once you have

created an account and logged in, click your name/photo on the top-right of the page and then navigate to "User Settings". Once on the Settings page, navigate to the end of the page and look for a sub-section named "API Keys" under the "Danger Zone" section. Click the "New Key" button to create the key and copy the key generated somewhere, as you will need it in your Python code to allow it to access your `wandb` account.

Similarly, you also need to create an account on HuggingFace and generate an API key/token, which you can do in Chapter 2. You can reuse the same token here and for all the following chapters. Listing 6-8 shows the code for using RL SB3 Zoo for training an agent, playing/uploading trained agent performance.

You first log in to your `wandb` account so that you can log the experiment results there. Next, you run `rl_zoo3.train` on the command line with various parameters to train and track the agents. Most of the command-line parameters are self-explanatory based on their names. The command-line parameters are over and above what can be seen in Listing 6-8. You can refer to RL SB3 Zoo documentation for more details.[3] Sometimes, we have found it easier to refer to the script code to understand all the parameters. For example, the `Train` script code can be browsed in GitHub at the location given in the reference.[4]

Once you have trained the agent, you evaluate it using an `rl_zoo3.enjoy` call and record the video with the help of `rl_zoo3.recrod_video`. Finally, `rl_zoo3.push_to_hub` can be used to push the trained agent weights, video, and other relevant details to HuggingFace.

Listing 6-8. End-to-end DQN with SB3 Zoo from `6.a-dqn-pytorch.ipynb`

```
# login to wandb
import wandb
wandb.login()

# Train the agent
!python -m rl_zoo3.train --algo dqn --env CartPole-v1 --save-freq 10000 \
--eval-freq 10000 --eval-episodes 10 --log-interval 400 --progress \
--track --wandb-project-name dqn-cartpole -f logs/6_a/rlzoo3/

# Evaluate trained agent
```

[3] https://rl-baselines3-zoo.readthedocs.io/en/master/

[4] https://github.com/DLR-RM/rl-baselines3-zoo/blob/master/rl_zoo3/train.py

```
!python -m rl_zoo3.enjoy --algo dqn --env CartPole-v1 --no-render
--n-timesteps 5000 --folder logs/6_a/rlzoo3

# Record a video
!python -m rl_zoo3.record_video --algo dqn --env CartPole-v1 --exp-id 0 -f
logs/6_a/rlzoo3/ -n 1000

# Login into Hugging face if not done so earlier
# To log to our Hugging Face account to be able to upload models to
the Hub.
from huggingface_sb3 import load_from_hub, package_to_hub, push_to_hub
from huggingface_hub import notebook_login

notebook_login()
!git config --global credential.helper store

# Share on HuggingFace
!python -m rl_zoo3.push_to_hub --algo dqn --env CartPole-v1 --exp-id 0 \
--folder logs/6_a/rlzoo3 --n-timesteps 1000 --verbose 1 --load-best  \
--organization nsanghi --repo-name dqn-cart-pole-rlzoo -m "Push to Hub"
```

Hyperparameter Optimization**

You have seen in the previous examples that there are a lot of hyperparameters that can impact the performance of the model. On a real-life RL training project, you are likely to spend a lot of time fine-tuning many of these parameters. First-time readers can skip this section and come back later as and when the need arises. This is not an essential read for understanding the RL algorithms.

RL SB3 Zoo is integrated with Optuna.[5] Optuna offers three key features 1) automated search for optimal hyperparameters using Python conditionals, loops, and syntax, 2) efficiently search large spaces and prune unpromising trials for faster results, and 3) parallelize hyperparameter searches over multiple threads or processes without modifying code. I cover high-level concepts of how Optuna works with RL SB3 Zoo. For more details, refer to the documentation of Optuna.

[5] https://optuna.readthedocs.io/en/stable/tutorial/index.html

To optimize, you first need to define the range over which you want to tune hyperparameters. You also define how these values will change over the range—for example, will they change by an integer value, or will the change be a float number, and will the change follow a log domain where the parameter values are sampled from log domain. A hyperparameter changing values over 0.1, 1, 10, 100, 1000 are equally spaced when you take the log. The previous range in base-10 log works out to -1, 0, 1, 2, 3, which is equally spaced. Log space sampling is the preferred choice when varying the learning rate α. The change could also be a categorical, meaning there is a defined set of values that it can take—for example, `batch_size` = [8, 16, 32, 64]. In RL SB3 Zoo, there is a file named `rl_zoo3/hyperparams_opt.py`[6] that provides a suggested list of tunable hyperparameters along with the ranges for each of the hyperparameters. The list and range are defined for each RL algorithm separately. Listing 6-9 shows the setting for DQN.

The hyperparameter sampler takes an argument `trial` of the type `optuna.Trial`, which defines a single instance of the run. For a given instance of the experiment run, you choose a specific set of hyperparameter values and return the same back for consumption by the rest of the optimization pipeline. The key part is that hyperparameter value sampling is done by the calls to `trial.suggest()` and the algorithm used for sampling is determined by the `--sampler` command-line argument while calling `rl_zoo3.train`. There are three possible values: `random`, `tpe`, and `skopt`. Random is a random search and can be a fairly good option in most cases. TPE stands for Tree-structured Parzen estimator algorithm. This sampler is based on independent sampling. On each trial, for each parameter, TPE fits one Gaussian Mixture Model (GMM) `l(x)` to the set of parameter values associated with the best objective values, and another GMM `g(x)` to the remaining parameter values. It chooses the parameter value `x` that maximizes the ratio `l(x)/g(x)`.[7] Scikit-Optimize, or `skopt`, is a simple and efficient library that minimizes (very) expensive and noisy black-box functions. It implements several methods for sequential model-based optimization. `skopt` aims to be accessible and easy to use in many contexts.[8]

[6] https://github.com/DLR-RM/rl-baselines3-zoo/blob/master/rl_zoo3/hyperparams_opt.py

[7] https://proceedings.neurips.cc/paper_files/paper/2011/file/86e8f7ab32cfd12577bc2619bc635690-Paper.pdf

[8] https://scikit-optimize.github.io/stable/

Listing 6-9. Hyperparameter Search Space for DQN from hyperparams_opt.py

```
def sample_dqn_params(trial: optuna.Trial, n_actions: int, n_envs: int,
additional_args: dict) -> Dict[str, Any]:
    """
    Sampler for DQN hyperparams.

    :param trial:
    :return:
    """
    gamma = trial.suggest_categorical("gamma", [0.9, 0.95, 0.98, 0.99,
    0.995, 0.999, 0.9999])
    learning_rate = trial.suggest_float("learning_rate", 1e-5, 1, log=True)
    batch_size = trial.suggest_categorical("batch_size", [16, 32, 64, 100,
    128, 256, 512])
    buffer_size = trial.suggest_categorical("buffer_size", [int(1e4),
    int(5e4), int(1e5), int(1e6)])
    exploration_final_eps = trial.suggest_float("exploration_final_
    eps", 0, 0.2)
    exploration_fraction = trial.suggest_float("exploration_
    fraction", 0, 0.5)
    target_update_interval = trial.suggest_categorical("target_update_
    interval", [1, 1000, 5000, 10000, 15000, 20000])
    learning_starts = trial.suggest_categorical("learning_starts", [0,
    1000, 5000, 10000, 20000])

    train_freq = trial.suggest_categorical("train_freq", [1, 4, 8, 16, 128,
    256, 1000])
    subsample_steps = trial.suggest_categorical("subsample_steps", [1,
    2, 4, 8])
    gradient_steps = max(train_freq // subsample_steps, 1)

    net_arch_type = trial.suggest_categorical("net_arch", ["tiny", "small",
    "medium"])

    net_arch = {"tiny": [64], "small": [64, 64], "medium": [256, 256]}[net_
    arch_type]
```

```
    hyperparams = {
        "gamma": gamma,
        "learning_rate": learning_rate,
        "batch_size": batch_size,
        "buffer_size": buffer_size,
        "train_freq": train_freq,
        "gradient_steps": gradient_steps,
        "exploration_fraction": exploration_fraction,
        "exploration_final_eps": exploration_final_eps,
        "target_update_interval": target_update_interval,
        "learning_starts": learning_starts,
        "policy_kwargs": dict(net_arch=net_arch),
    }

    if additional_args["using_her_replay_buffer"]:
        hyperparams = sample_her_params(trial, hyperparams, additional_
        args["her_kwargs"])

    return hyperparams
```

The other important part of the hyperparameter optimization is the concept of pruning, which refers to the strategy for the termination of unpromising trials. In some literature, it is also known as early stopping. In SB3 Zoo, pruning is controlled via the `--prunner` command-line argument, which can take one of the three possible values: `halving`, `median`, or `none`. Successive halving, or halving, has a simple idea behind the algorithm. Given an input budget, uniformly allocate the training budget to a set of arms (number of parameter combinations) for a predefined number of iterations, evaluate their performance, throw out the worst half, and repeat until just one hyperparameter configuration remains. You can read more about the concept of pruning in the paper titled "Non-stochastic Best Arm Identification and Hyperparameter Optimization" by Kevin Jamieson and Ameet Talwalkar.[9] Median pruning refers to the strategy that a trial is pruned if the trial's best intermediate result is worse than the median of the intermediate results of previous trials at the same step.

[9] https://arxiv.org/abs/1502.07943

`rl_zoo.train` provides a few other controls on the optimization. The `--n-startup-trials` command-line argument defines the number of trials before the Optuna sampler starts. The `--n-trials` command-line argument defines the number of trials for optimizing hyperparameters. The `--max-total-trials` command-line argument defines the number of (potentially pruned) trials for optimizing hyperparameters. This last argument applies to the entire optimization process and takes precedence over `--n-trials`. There are a few more and you can read about them in the `rl_zoo3.train` script.[10]

Not to forget—the most important command-line argument is `--optimize`. It launches `rl_zoo3.train` in hyperparameter optimization mode.

Lastly, there are many ways to specify `hyperparams_opt.py`. Listing 6-10 shows a sample call of `rl_zoo.train` with DQN and the `CartPole` environment.

Listing 6-10. Hyperparameter Optimization with rl_zoo3.train from `6.a-dqn-pytorch.ipynb`

```
!python -m rl_zoo3.train --algo dqn --env CartPole-v1 --save-freq 10000 \
--eval-episodes 10 --log-interval 100 --progress --track \
--wandb-project-name dqn-cartpole \
--optimization-log-path logs/optimization/ --log-folder logs --optimize-
hyperparameters --n-jobs 2 \
--study-name dqn-test --max-total-trials 25
```

Note that, based on the settings, it may take a long time to complete the optimization run. Moving on to plotting the optimization results, the `rl_zoo3/plots` and `scripts` folders in the `rl_zoo3` repository provide a few scripts to plot the results. In the `scripts` folder, `parse_study.py` helps parse an optimization run and saves the *n* best hyperparameter configuration. Listing 6-11 shows an example of running this script after the optimization run in Listing 6-10 completes.

Listing 6-11. Saving "n" Best Configurations After an Optimization Run from the `6.a-dqn-pytorch.ipynb` notebook

```
!python -m rl_zoo3.scripts.parse_study -i ./logs/dqn/report_CartPole-
v1_500-trials-50000-tpe-median_1698290229.pkl --print-n-best-trials 10 \
--save-n-best-hyperparameters 10 --folder ./logs/dqn/6.a/best-params/
```

[10] https://github.com/DLR-RM/rl-baselines3-zoo/blob/master/rl_zoo3/train.py

Integration with Rliable library(**)

In 2021, a paper called "Deep Reinforcement Learning at the Edge of the Statistical Precipice" by Agarwal et al.,[11] received the Outstanding award at NeurIPS 2021. To quote from the paper:

> *"Deep reinforcement learning (RL) algorithms are predominantly evaluated by comparing their relative performance on a large suite of tasks. Most published results on deep RL benchmarks compare point estimates of aggregate performance such as mean and median scores across tasks, ignoring the statistical uncertainty implied by the use of a finite number of training runs...."*

The authors go on to say that the shift toward computationally demanding benchmarks, especially with benchmarks based on environments like ALE (Arcade Learning Environments), has increasingly led to a practice of evaluating only on a small number of tasks, which is worsening the statistical uncertainty of the point estimates. The authors illustrate this point using ALE and then go on to recommend use of

> *"...interval estimates of aggregate performance and propose performance profiles to account for the variability in results, as well as present more robust and efficient aggregate metrics, such as interquartile mean scores, to achieve small uncertainty in results."*

You can read more about it in the link to the blog accompanying the paper. The authors also released a Python library called `rliable`, which has been integrated into RL SB3 Zoo. There are three key scripts provided by SB3 Zoo for plotting. `scripts/all_plots.py`, `scripts/plot_from_file.py` for plotting evaluations, and `scripts/plot_train.py` for plotting training reward/success. The `6.a-dqn-pytorch.ipynb` notebook contains some samples.

First, you need to do multiple training runs to collect diverse data points for meaningful plotting. For this, you can rerun the training script from Listing 6-8 multiple times, with some random variations of the hyperparameters. You'll create various training trajectories and have multiple validation results. Once you have had enough runs, the next step is to prepare the results file. Listing 6-12 contains the code for running these and other plotting scripts. The first command line in Listing 6-12 is about preparing the file for all plots. It requires parameters like the algorithm, the environment,

[11] https://agarwl.github.io/rliable/

the location where the data from the training runs is stored, and the output location where you want to store file generated by this script. After the results have been saved in a file, you feed the same into the second command, as shown in Listing 6-12. `plot_from_file` takes as input the file produced from the previous `all_plots` command and, based on the command-line parameters, it uses `rliable` or other graphing libraries to create the graphs. Some of the command-line arguments pertain to the `rliable` library. You may need to explore the `rliable` documentation for a better understanding of what they control and what is the significance of each of them. You can look into the Plotting section of SB3 Zoo documentation to understand the various command-line options these scripts have. Like `all_plots`, there is another script called `plot_train` that plots the training graphs.

Listing 6-12. Plotting from Notebook `6.a-dqn-pytorch.ipynb`

```
!python -m rl_zoo3.plots.all_plots --algo dqn --env CartPole-v1 \
-f logs/6_a/rlzoo3/ -o logs/6_a/rlzoo3/dqn_results

!python -m rl_zoo3.plots.plot_from_file -i logs/6_a/rlzoo3/dqn_
results.pkl \
-latex -l DQN --output logs/6_a/rlzoo3/dqn_test_plot.svg

!python -m rl_zoo3.plots.plot_from_file -i logs/6_a/rlzoo3/dqn_
results.pkl \
--rliable --versus --iqm --boxplot \
-latex -l DQN --output logs/6_a/rlzoo3/dqn_test1_plot.svg
```

This completes the implementation of a full DQN to train an agent. The core idea was to focus on the algorithm and teach you how to write a DQN learning agent. You will now use the same implementation with minor tweaks so that the agent can play Atari games using the game image pixel values as the state. Note that you will be building a more powerful neural network and will need to train agents over many more time steps running into a million steps. Therefore, unless you have access to a GPU locally, you may consider running the Atari DQN notebook on Google Colab. The notebook contains commands for all the package installations required to run the notebook on Google Colab and is thus self-contained.

Atari Game-Playing Agent Using DQN

In a 2013 seminal paper titled "Playing Atari with Deep Reinforcement Learning,"[12] the authors used a deep learning model to create a neural network–based Q-learning algorithm. They christened it Deep Q Networks. This is exactly what you implemented in the previous section. You will now briefly learn about the additional steps the authors took to train the agent to play Atari games. It is mostly the same as the previous section with two key differences: 1) uses game image pixel values as state inputs that necessitate some preprocessing, and b) uses convolutional networks inside an agent instead of linear layers that you saw in the previous section. The rest of the approach to calculate the loss L and carry out the training remains the same. Note that training with convolution networks takes a lot of time, especially on regular PCs/laptops. Get ready to watch the training code run for hours, even on moderately powerful GPU-based machines.

You can find the complete code to train the agent in PyTorch in the Python notebook called `6.c-dqn_atari_pytorch.ipynb`. The Gymnasium library has implemented many of the transformations needed for an Atari game image, and wherever possible, these examples use the same.

Atari Environment in Gymnasium

Let's first talk about Atari games environment as provided within Gymnasium and also the image preprocessing done to get the image pixel values ready for feeding into the deep learning network. Gymnasium provides a set of Atari 2600 environments simulated through Stella and the Arcade Learning Environment.

The Atari 2600 Video Computer System (VCS), introduced in 1977, was the most popular home video game system of the early 1980s. You can enjoy all the popular Atari 2600 games on your PC thanks to Stella. Stella is a multi-platform Atari 2600 VCS emulator released under the GNU General Public License (GPL).

The Arcade Learning Environment (ALE) is a simple framework that allows researchers and hobbyists to develop AI agents for Atari 2600 games. It is built on top of the Atari 2600 emulator called Stella and separates the details of emulation from agent design. ALE provides Python bindings and supports OpenAI Gym and Farma Gymnasium.

[12] https://arxiv.org/pdf/1312.5602.pdf

ALE-py doesn't include the Atari ROMs (`pip install gymnasium[atari]`), which are necessary to make Atari environments in the Gymnasium `make` function. The Python installation instructions in the beginning of the book take care of this.

The action space of Atari games is comprised of 18 actions numbered from 0 to 17. By default, most environments use a smaller subset of the legal actions, excluding those actions that don't have an effect in that specific game. If users are interested in using all possible actions, pass the keyword argument `full_action_space=True` to `gymnasium.make`. I will not be using this as the neural networks will be used to output only relevant actions for the specific game in question.

Atari games have three types of observation spaces: the most common one is 1) RGB image of size 210x160 with type `Box(0, 255, (210, 160, 3), np.uint8)`, 2) the grayscale version of the image, and 3) the RAM state of 128 bytes. You can control this using the `obs_type` argument to the `gym.make` function call.

As the Atari games are entirely deterministic, agents could achieve state-of-the-art performance by simply memorizing an optimal sequence of actions while completely ignoring observations from the environment. There are several methods to avoid this. Quoting from the Gymnasium documentation:

- *Sticky actions:* Instead of always simulating the action passed to the environment, there is a small probability that the previously executed action is used instead. In the v0 and v5 environments, the probability of repeating an action is 25% while in v4 environments, the probability is 0%. Users can specify the repeat action probability using `repeat_action_probability to gymnasium.make`.
- *Frameskipping*: On each environment step, the action can be repeated for a random number of frames. This behavior may be altered by setting the `argument frameskip` keyword to either a positive integer or to a tuple of two positive integers. If `frameskip` is an integer, frame skipping is deterministic, and in each step the action is repeated `frameskip` many times. Otherwise, if `frameskip` is a tuple, the number of skipped frames is chosen uniformly at random between `frameskip[0]` (inclusive) and `frameskip[1]` (exclusive) in each environment step.

Some games allow users to set a difficulty level and a game mode. Different modes/difficulties may have different game dynamics and different action spaces. This example does not vary either of these. It uses the default of 0.

You will be implementing an agent for the game of Breakout. In Breakout, there is a paddle at the bottom, and the idea is to move the paddle to ensure the ball does not fall below it. The player needs to use the paddle to hit the ball and take out as many bricks as possible. Each time the ball misses the paddle, the player loses a life. The player has five lives to start with. Breakout has an action space of 0-3 `['NOOP', 'FIRE', 'RIGHT', 'LEFT']` with 0: "No Operation (NOOP), 1: FIRE to start the game, 2: move paddle right and 3: move paddle left. You can refer to the Gymnasium documentation for more details.[13] Figure 6-3 shows three frames of the game.

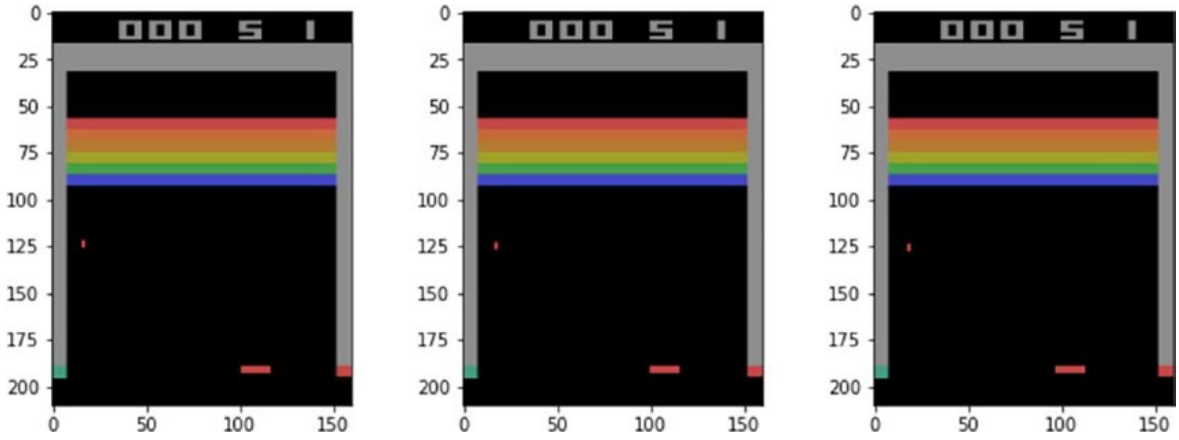


Figure 6-3. *Atari Breakout game images*

The Atari game images are 210×160-pixel images with a 128-color palette. Listing 6-13 shows the code used to create the environment and print a few frames of the game.

Listing 6-13. Creating an Atari Breakout Game Instance from notebook `6.c-dqn_atari_pytorch.ipynb`

```
def make_env(env_name, frameskip=5, repeat_action_probability=0.25, render_
mode='rgb_array', mode=0, difficulty=0):
    # remove time limit wrapper from environment
    env = gym.make(env_name,
                   render_mode=render_mode,
                   frameskip=frameskip,
                   repeat_action_probability=repeat_action_probability,
                   mode=mode,
```

[13] https://gymnasium.farama.org/environments/atari/breakout/

```
                    difficulty=difficulty
                   ).unwrapped
    return env
env_name = "ALE/Breakout-v5"

env = make_env(env_name)
env.reset(seed=127)

n_cols = 4
n_rows = 2
fig = plt.figure(figsize=(16, 9))

for row in range(n_rows):
    for col in range(n_cols):
        ax = fig.add_subplot(n_rows, n_cols, row * n_cols + col + 1)
        ax.imshow(env.render())
        env.step(env.action_space.sample())
plt.show()
```

Preprocessing and Training

You will do preprocessing to prune down the image size and make the convolutional network run faster. You also scale down the images. You can convert the images to grayscale again to reduce the size of the input vector, with one channel of grayscale instead of three channels of colors for RGB (the red, green, and blue channels). A preprocessed single frame of the image of size (1×84×84—channel, width, height) in PyTorch just gives the static state. The position of the ball or paddle does not tell you about the direction in which both are moving. Accordingly, you will stack a few consecutive frames of the game images together to train the agent. You will stack four reduced-size grayscale images that will the state S feeding into the neural network. The input (i.e., state S) will be of size 4×84×84 in PyTorch and 84×84×4 in TensorFlow, where 4 refers to the four frames of the game images and 84×84 is the grayscale image size of each frame. Stacking four frames together will allow the agent network to infer the direction of movement of the ball and paddle.

Listing 6-14 shows the code to create the environment, including the preprocessing steps. When you plot a single observation of the environment after the preprocessors, you see that state/observation has a shape of 4x84x84. In the `make_env` function, you first

call gym.make with frameskip=1 which is the default behavior, meaning that the next call to step will produce the next game frame. The reason you do this is because you will be wrapping this with additional wrappers, which will take care of stacking multiple frames together. Therefore, you do not want any frame skipping in the call to gym.make. Note that all along you import Gymnasium as gym. Accordingly, in this code I refer to the package as Gymnasium or Gym interchangeably, for example, gymnasium.make or gym.make. Both refer to the Gymnasium library and calling the make function.

Next, you wrap the Gymnasium Breakout environment with AtariPreprocessing to set screen_size to 84 pixels, and the pixel values are mapped to 0-1 range instead of 0-255 range with the help of scale_obs=True. The complete list of the arguments to AtariPreProcessing can be seen in the Gymnasium documentation.[14] The same documentation page also provides details with respect to many other wrappers that can be used to wrap the vanilla environments to augment or alter the default behaviors. You will use the default value of frame_skip=4, as you need to stack four frames together to provide the context of ball and pad direction before feeding the observations to the neural network. Therefore, the next wrapper you use is FrameStack with num_stack=4 to stack four frames of game images together. Finally, based on the clip_rewards flag, you further wrap the environment with the TransformReward wrapper. This wrapper takes the input environment and a function to transform the reward. In this case, you use a simple lambda function lambda r: np.sign(r) to return just the sign of the reward, dropping the magnitude. This is in line with the process followed in the original paper.

Listing 6-14. Creating an Atari Breakout Game Instance with Required Wrappers to Train the Agent from notebook 6.c-dqn_atari_pytorch.ipynb

```
from gymnasium.wrappers import AtariPreprocessing
from gymnasium.wrappers import FrameStack
from gymnasium.wrappers import TransformReward

def make_env(env_name,
             clip_rewards=True):

    env = gym.make(env_name,
                   render_mode='rgb_array',
                   frameskip=1
```

[14] https://gymnasium.farama.org/api/experimental/wrappers/

```
            )
    env = AtariPreprocessing(env, screen_size=84, scale_obs=True)
    env = FrameStack(env, num_stack=4)
    if clip_rewards:
        env = TransformReward(env, lambda r: np.sign(r))
    return env

env = make_env(env_name)
obs, _ = env.reset()
n_actions = env.action_space.n
state_shape = env.observation_space.shape

print("Observation Shape:", state_shape) #prints 4x84x84


obs = obs[:] #unpack lazyframe
obs = np.transpose(obs,[1,0,2]) #move axes
obs = obs.reshape((obs.shape[0], -1))

plt.figure(figsize=[15,15])
plt.title("Agent observation (4 frames left to right)")
plt.imshow(obs, cmap='gray')
plt.show()
```

The previous preprocessing step produces the final state that you will be feeding into the network. This is 4×84×84 in PyTorch, where 4 refers to the four frames of the game images and 84×84 is the grayscale image size of each frame. Figure 6-4 shows the observation as produced by the code in Listing 6-14 and which is used as input to the neural network.

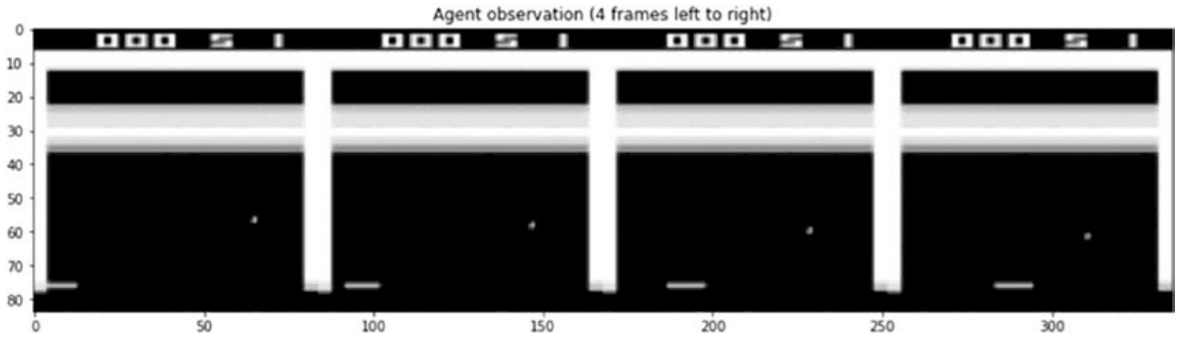


Figure 6-4. *Processed image to be used as state input into the neural network*

Next, you build the neural network that will take in the previous image, that is, the state/observation *S*, and produce q-values for all four actions in this case. The shape of the network you will build is given here:

- Input: PyTorch tensor of shape [batch_size, 4, 84, 84]
- First hidden convolutional layer: 16 nos of 8x8 filters with stride 4 and ReLU activation producing output of shape [batch_size, 16, 20, 20]
- Second hidden convolution layer: 32 nos of 4x4 filters with stride of 2 and ReLU activation producing output of shape [batch-size, 32, 9, 9]
- Flatten the observation to shape [batch, 32x9x9] = [batch_size, 2592]
- Third hidden fully connected linear layer with 256 outputs and ReLU activation producing output of shape [batch_size, 256]
- Output layer is a linear layer with "n_actions" units with no activation producing output of shape [batch_size, 4]

You implement the usual functions—`forward`, `get_qvalues`, and `sample_actions`. These are similar to what you did in the `6.a-dqn_pytorch.ipynb` notebook for the `CartPole` environment. Listing 6-15 shows the code implementing what you just read about when creating a `DQNAgent` specific to the Atari game in question.

Listing 6-15. DQNAgent Implementation from Notebook `6.c-dqn_atari_pytorch.ipynb`

```
class DQNAgent(nn.Module):
    def __init__(self, state_shape, n_actions, epsilon=0):

        super().__init__()
        self.epsilon = epsilon
        self.n_actions = n_actions
        self.state_shape = state_shape

        state_dim = state_shape[0]
        # a simple NN with state_dim as input vector (input is state s)
        # and self.n_actions as output vector of logits of q(s, a)
        self.network = nn.Sequential()
```

```
        self.network.add_module('conv1', nn.Conv2d(4,16,kernel_size=8,
        stride=4))
        self.network.add_module('relu1', nn.ReLU())
        self.network.add_module('conv2', nn.Conv2d(16,32,kernel_size=4,
        stride=2))
        self.network.add_module('relu2', nn.ReLU())
        self.network.add_module('flatten', nn.Flatten())
        self.network.add_module('linear3', nn.Linear(2592, 256)) #2592
        calculated above
        self.network.add_module('relu3', nn.ReLU())
        self.network.add_module('linear4', nn.Linear(256, n_actions))

        self.parameters = self.network.parameters

    def forward(self, state_t):
        # pass the state at time t through the network to get Q(s,a)
        qvalues = self.network(state_t)
        return qvalues

    def get_qvalues(self, states):
        # input is an array of states in numpy and outout is Qvals as
        numpy array
        states = torch.tensor(states, device=device, dtype=torch.float32)
        qvalues = self.forward(states)
        return qvalues.data.cpu().numpy()

    def sample_actions(self, qvalues):
        # sample actions from a batch of q_values using epsilon
        greedy policy
        epsilon = self.epsilon
        batch_size, n_actions = qvalues.shape
        random_actions = np.random.choice(n_actions, size=batch_size)
        best_actions = qvalues.argmax(axis=-1)
        should_explore = np.random.choice(
            [0, 1], batch_size, p=[1-epsilon, epsilon])
        return np.where(should_explore, random_actions, best_actions)
```

This is then followed by the usual implementation of experience replay, functions to play and record the game, creating an initial copy of target network as a deep copy of agent network, the `compute_td_loss` function to compute TD loss, instantiating all the machinery of agent, target network, and replay buffer.

Once these are done, you fill the replay buffer with an initial set of actions by having the agent randomly choose actions. You then go on to create the hyperparameters, as shown in Listing 6-16.

At this point, the agent is ready to be trained. Note that to get meaningful behavior, you need to train the agent close to one million steps or so. The network is also substantially more complex than the one used in the `CartPole` environment. Therefore, it takes a long time to train the Atari game playing agent for one million steps. You are advised to run the code on Google Colab or run it on a local computer that has GPU computing capabilities.

Listing 6-16. Hyperparameters from Notebook `6.c-dqn_atari_pytorch.ipynb`

```
#set up some parameters for training
timesteps_per_epoch = 1
batch_size = 32

total_steps = 100
# total_steps = 3 * 10**6
# We will train only for a sample of 100 steps
# To train the full network on a CPU will take hours.
# in fact even GPU training will be fairly long
# Those who have access to powerful machines with GPU could
# try training it over 3-5 million steps or so

#init Optimizer
opt = torch.optim.Adam(agent.parameters(), lr=1e-4)

# set exploration epsilon
start_epsilon = 1
end_epsilon = 0.05
eps_decay_final_step = 1 * 10**6

# setup some frequency for logging and updating target network
loss_freq = 20
```

```
refresh_target_network_freq = 100
eval_freq = 1000

# to clip the gradients
max_grad_norm = 5000
```

The code for training the agent is similar to what you saw for `CartPole` in the `6.a-dqn_pytorch.ipynb notebook`. The code for recording and playing the video of trained agent, pushing the trained agent parameters and video to HuggingFace and so on, are pretty similar as before. Therefore, I do not explicitly show the listings of the code here. Interested readers may refer to the `6.c-dqn_atari_pytorch.ipynb` notebook. You can also train the same agent with Stable Baselines3 (SB3) or RL SB3 Zoo. You are encouraged to write that code using `CartPole` notebook code as a starting point. The key difference you will have to watch out for is the preprocessing of Breakout environment with specific Atari wrappers, as discussed earlier in this section. The default policy also must be switched from `MLPPolicy` to `CnnPolicy`, either with default values or with custom network as per your needs.

This completes the implementation and training of DQN for the Atari game. Now that you know how to train a DQN agent, in Chapter 7, an optional chapter, you will investigate some issues and various approaches you could take to modify the DQN. I close the chapter by reviewing the various other environments that are integrated with Gymnasium and/or Stable Baselines3, including the kind of problems they solve and the technical considerations for using these environments.

Overview of Various RL Environments and Libraries

Until now in this book, you have looked at Gymnasium and its earlier version OpenAI Gym. They both, barring some minor differences as discussed in the beginning of this chapter, specify a standard interface for defining RL environments—the need for such environments to have functions like `reset, step, render,` and so on.

You also looked at specific environments like Cart Pole, Mountain Car, Lunar Lander, and Atari Breakout. These environments follow the specifications of Gymnasium/Gym and are made available inside the Gymnasium library. There are many other third-party, open-source environments for different use cases which are either integrated within Gymnasium by the group maintaining the Gymnasium codebase or by other open-source libraries. To help you appreciate that RL is used not just for playing games but for

very large and varied scenarios, the next section look at some of the other environments. What I discuss next is by no means exhaustive, rather just a sample. Readers, based on their interest, may decide to dive into the details of libraries/environments.

PyGame

PyGame is a Python-based library. This library includes several modules for playing sound, drawing graphics, handling mouse inputs, and so on. Many RL environments in gymnasium use PyGame to implement rendering logic in environments for human and/or visual space representation of the game state.

PyGame is not just for RL. It is a wider library that's used for implementing stand-alone games and interactive environments using the Python programing language. You can possibly take a full course on PyGame to understand the details; it has an extensive set of features.

As an RL engineer, you will not be not needing to dive into the details. Even if you design your customer environments, you will use other higher-level libraries that may internally use PyGame to implement the graphics.

MuJoCo

MuJoCo stands for Multi-Joint dynamics with Contact. It is a physics engine for facilitating research and development in robotics, biomechanics, graphics and animation, and other areas where fast and accurate simulation is needed. The unique dependencies for this set of environments can be installed via `pip install gymnasium[mujoco]`.

Gymnasium has eleven MuJoCo environments: Ant, HalfCheetah, Hopper, Humanoid, HumanoidStandup, InvertedDoublePendulum, InvertedPendulum, Pusher, Reacher, Swimmer, and Walker2d. A sample screenshot of these environments is shown in Figure 6-5.

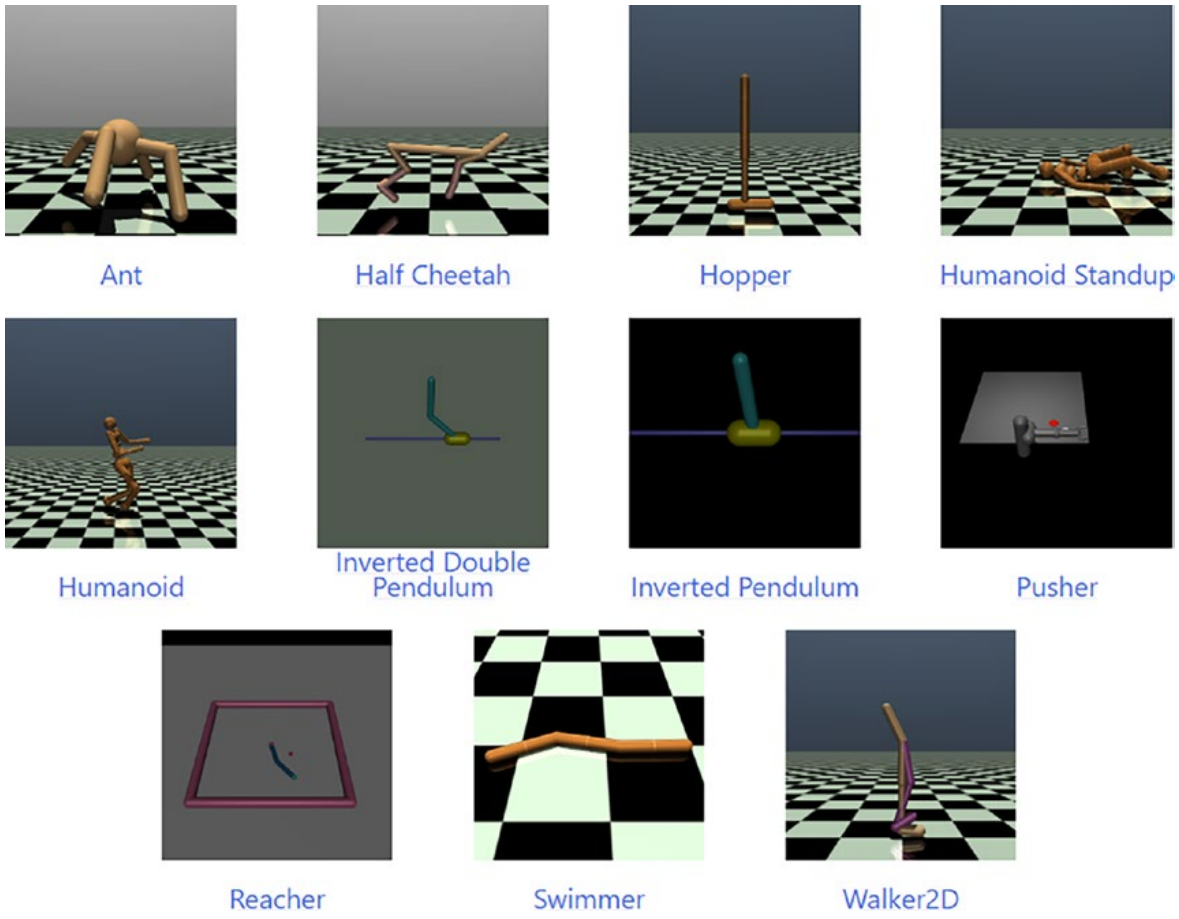


Figure 6-5. *MuJoCo environments in Gymnasium*

All of these environments are stochastic in terms of their initial state, with Gaussian noise added to a fixed initial state in order to add stochasticity. The state spaces for MuJoCo environments in Gymnasium consist of two parts that are flattened and concatenated together: a position of a body part (`mujoco-py.mjsim.qpos`) or joint and its corresponding velocity (`mujoco-py.mjsim.qvel`). Often, some of the first positional elements are omitted from the state space since the reward is calculated based on their values, leaving it up to the RL algorithm to infer those hidden values indirectly.

If you want to use these environments from gymnasium, you also need to install the MuJoCo engine. DeepMind acquired MuJoCo in late 2021 and open-sourced the code, making it free for everyone. Instructions on installing the MuJoCo engine can be found on the website[15] and on the GitHub repository.[16]

[15] https://mujoco.org/

[16] https://github.com/google-deepmind/mujoco

Using MuJoCo with Gymnasium also requires the `mujoco` framework be installed (this dependency is installed with the previous `pip` command). Environments can be configured by changing the XML files or by tweaking the parameters of their classes.

Similar to Atari, you can use RL Zoo to train the agent. Listing 6-17 shows the terminal command to train the agent. The complete code with the required installation steps can be found in the `6.d-robotic-env.ipynb` notebook.

Listing 6-17. Training the MuJoCo Environment 6.d-robotic-env.ipynb

```
!python -m rl_zoo3.train --algo ppo --env Reacher-v4 --progress \
--log-interval 400 --save-freq 10000 --eval-freq 10000 --eval-episodes 10 \
--n-timesteps 100000 --conf-file ppo_config_6e.py \
--track --wandb-project-name ppo-reacher-v4 -f logs/6_e/rlzoo3/
```

Unity ML Agents

The Unity Machine Learning Agents Toolkit (ML-Agents[17]) is an open-source project that enables games and simulations to serve as environments for training intelligent agents. ML-Agents provides PyTorch-based implementations of state-of-the-art algorithms to enable game developers and hobbyists to easily train intelligent agents for 2D, 3D and VR/AR games. Researchers can also use the provided simple-to-use Python API to train agents using reinforcement learning, imitation learning, neuro evolution, or any other methods. These trained agents can be used for multiple purposes, including controlling non-playing character (NPC) behavior (in a variety of settings such as multi-agent and adversarial), automated testing of game builds, and evaluating different game design decisions pre-release.

The `ML-agents` toolkit comes with 17+ sample Unity environments. It has support for training single-agent, multi-agent cooperative, and multi-agent competitive scenarios via several deep reinforcement learning algorithms. You can very easily add your own customer environment or custom training algorithm.

It also provides a way to wrap Unity Learning environments as a OpenAI Gym environment for you to use it in your code as a straight drop-in in place of any other environment. You can read about it in the documentation for `ml-agents` in GitHub, the link for the same is referenced at the beginning of this sub-section. Unfortunately, as of

[17] https://github.com/Unity-Technologies/ml-agents

writing this book, `ml-agents` supports the OpenAI gym API version and not the latest API version from Gymnasium and there is no clear direction from the maintainers of the code on the migration from Gym to Gymnasium.

You can refer to the `ml-agent` tutorials or go to the https://huggingface.co/docs/hub/ml-agents link, which walks through the way `ml-agents` can be used in HuggingFace ecosystem.

PettingZoo

PettingZoo is a simple, Pythonic interface capable of representing general *multi-agent reinforcement learning* (MARL) problems. PettingZoo includes a wide variety of reference environments, helpful utilities, and tools for creating your own custom environments. While I mention this library here for the sake of completeness and for putting together all environment related discussions in a single place in the book, I will be covering PettingZoo in greater detail in a subsequent chapter on multi-agent RL. Some common multi-agent environments include:

- Atari multiplayer games
- Board games like Chess and Go
- Card games like rummy
- Multi-Particle Environments (MPEs), where particle agents can move, communicate, see each other, push each other around, and interact with fixed landmarks
- The SISL environments, which are a set of three cooperative multi-agent benchmark environments, created at SISL (Stanford Intelligent Systems Laboratory) and released as part of "Cooperative multi-agent control using deep reinforcement learning"

Bullet Physics Engine and Related Environments

Bullet is a physics engine which simulates collision detection as well as soft and rigid body dynamics. It has been used in video games and for visual effects in movies. Erwin Coumans, its main author, won a Scientific and Technical Academy Award[4] for his work on Bullet.[18]

[18] https://en.wikipedia.org/wiki/Bullet_%28software%29

PyBullet is a fast and easy to use the Python module for robotics simulation and machine learning, with a focus on sim-to-real transfer. PyBullet wraps the Bullet API. PyBullet can be easily used with TensorFlow and OpenAI Gym. Researchers from Google Brain, Stanford AI Lab, OpenAI, and many other labs use PyBullet.

Then come the various RL environments, which further wrap PyBullet to create custom RL environments that follow the OpenAI gym API signatures. Some of the common ones are as follows:

- `Pandas-gym`: PyBullet based simulations of a robotic arm moving objects.
- `Parallel Game engine (PGE)`: Intended for fast and pretty 3D artificial intelligence experiments. However, this open-source project has not seen any update since 2018 and may be considered stale.
- `PyFlyt`: UAV Flight Simulator Environments for Reinforcement Learning Research. It is a library for testing reinforcement learning algorithms on various UAVs. Built on the Bullet physics engine, it offers flexible rendering options, time-discrete steppable physics, Python bindings, and support for custom drones of any configuration, be it biplanes, quadcopters, rockets, and anything you can think of. It is still under development and supports the newer Gymnasium API signatures.

There are a few more that are mentioned in the Gymnasium documentation under section on third-party environments.

CleanRL

CleanRL is a deep reinforcement learning library that provides high-quality, single-file implementation with research-friendly features. The implementation is clean and simple. With single file implementation, every detail about an algorithm variant is put into a single stand-alone file. For example, `ppo_atari.py` has 340 lines of code and contains all the implementation details on how PPO works with Atari games, so it is a great reference implementation to read.

CleanRL only contains implementations of online deep reinforcement learning algorithms. Refer to `https://github.com/corl-team/CORL`, which shares a similar design philosophy as CleanRL but for offline RL algorithms.

CleanRL uses "Poetry" which is yet another Python packaging and dependency management tool and achieves a similar purpose as that of venv or conda. You can explore Poetry's dependency management and its pros and cons by visiting the Poetry documentation.[19]

Listing 6-18 shows the set of commands you can run to install Poetry, followed by CleanRL and then training an agent DQN. The code in Listing 6-18 has been tried on WSL2 Ubuntu on Windows 11 and should also work out-of-the-box on Ubuntu and macOS. For macOS, you may want to use the Homebrew package manager for installing Python and the other libraries.

Listing 6-18. Running DQN Using CleanRL

```
# to install python and pip followed by poetry

sudo apt install python3-pip
python3 -m pip install --user -U pipx
pipx install poetry

# install CleanRL and associated python packages
git clone https://github.com/vwxyzjn/cleanrl.git && cd cleanrl
poetry install

# if you get some error during the execution, you may want to install
specific version of SB3
poetry run pip install "stable_baselines3==2.0.0a1"

# to enter poetry shell which is like conda activate
poetry shell

# to login into wandb and huggingface
# You will need api-key/token from your wandb and huggingface accounts
wandb login
huggingface-cli login

# train CartPole with DQN implementation by CleanRL
# please use your HuggingFace name instead of "nsanghi" for parameter
"--hf-entity"
```

[19] https://python-poetry.org/

```
python -m cleanrl.dqn  --seed 1  --env-id CartPole-v1     --total-
timesteps 50000 --track --capture-video --save-model --upload-model --hf-
entity nsanghi

# See your results
# replace "nsanghi" with your hf-account name
https://wandb.ai/nsanghi/cleanRL/runs/99rex9ov
https://huggingface.co/nsanghi/CartPole-v1-dqn-seed1

# please look at clearn_rl/dqn.py in the github repo
# for more command line options and default values
# see https://docs.cleanrl.dev/get-started/installation/#prerequisitesmore
# for additional environments and different algos and backends

# to leave poetry shell
exit
```

MineRL

MineRL is a Python 3 library that provides a OpenAI Gym interface for interacting with the video game Minecraft, accompanied with datasets of human gameplay. Started as a research project at Carnegie Mellon University, MineRL aims to assist in the development of various aspects of artificial intelligence within Minecraft.

Refer to the documentation[20] for more details on how to install and use this. You can also refer to the Tutorial section of the documentation for examples on how to run MineRL on Colab.

FinRL

FinRL is an open-source framework for financial reinforcement learning. FinRL provides a framework that supports various markets, State of the Art (SOTA) deep learning RL (DRL) algorithms, benchmarks of many quant finance tasks, live trading, and so on.

The DRL framework is powerful in solving dynamic decision-making problems by learning through interactions with an unknown environment, thus exhibiting two major advantages: portfolio scalability and market model independence. Automated trading

[20] https://minerl.readthedocs.io/en/latest/index.html

is essentially making dynamic decisions, namely, to decide where to trade, at what price, and what quantity, over a highly stochastic and complex stock market. Taking many complex financial factors into account, DRL trading agents build a multi-factor model and provide algorithmic trading strategies, which are difficult for human traders. Interested readers can read more about it on GitHub.[21] The documentation also contains some notebooks showcasing on its use.

FlappyBird Environment

The GitHub repository `markub3327/flappy-bird-gymnasium`[22] contains the implementation of Gymnasium environment for the FlappyBird game.

FlappyBird, `FlappyBird-v0`, has 12 dimensional observation spaces with the following meanings:

1. The last pipe's horizontal position
2. The last top pipe's vertical position
3. The last bottom pipe's vertical position
4. The next pipe's horizontal position
5. The next top pipe's vertical position
6. The next bottom pipe's vertical position
7. The next next pipe's horizontal position
8. The next next top pipe's vertical position
9. The next next bottom pipe's vertical position
10. The player's vertical position
11. The player's vertical velocity
12. The player's rotation

The action space is discrete, with two values:

1. Do nothing - "0"
2. Flap - "1"

[21] https://github.com/AI4Finance-Foundation/FinRL

[22] https://github.com/markub3327/flappy-bird-gymnasium

And reward is comprised of:

+0.1 - Every frame it stays alive

+1.0 - Successfully passing a pipe

-1.0 - Dying

Interested readers can refer to the `6.e-dqn_flappy_pytorch.ipynb` notebook for training and evaluating the FlappyBird environment using DQN. It is very similar to what you saw for Atari DQN. This brings you to the end of this chapter.

Summary

This was a long chapter, where you looked at DQN in depth and the various RL environments. It started with a quick recap of Q-learning and the derivation of the update equation for DQN. You then looked at the implementation of DQN in both PyTorch and TensorFlow for a simple `CartPole` environment. You saw the complete code for the DQN algorithm.

You followed it with a review of the Gymnasium API and compared the changes it has undergone from its original version under OpenAI Gym. You looked at how Stable Baselines3 (SB3) and RL SB3 Zoo (SB3 Zoo) can be used for training agents, where you leverage the pre-implemented DQN and various other algorithms. A quick review of the process to record the video of a trained agent in action was also discussed.

The next two sections were on hyperparameter optimization using Optuna and SB3 and plotting using the Rliable library.

This completed the walkthrough of DQN and its associated concepts. Next, I talked about applying DQN to Atari games, which resulted in discussing preprocessing steps commonly required for games like these.

The final section was devoted to the various RL environments ,ranging from MuJoCo for simple joint robots, to Unity-based ML agents, and areas like finance under the FinRL library.